

Module 05

RAG Fundamentals

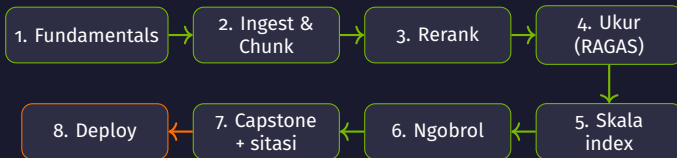
Ground → Ingest → Retrieve → Ukur → Skala → Ngobrol →
Sitasi → Deploy



8 notebook · gratis di Colab Tesla T4 16GB · Navasena AI · 2026

Peta Perjalanan

Dari satu jawaban sederhana sampai layanan RAG siap-deploy



Slide ini tidak per-notebook — kita susun per **konsep**. Tiap konsep adalah satu balok di peta ini.

Act 1

Kenapa RAG?

LLM pintar berbahasa, tapi bukan sumber kebenaran fakta

Dua Masalah LLM Polos

Kenapa LLM saja tidak cukup untuk menjawab fakta

- ▶ LLM menjawab dari **ingatan** hasil pelatihannya saja
- **Halusinasi** — mengarang jawaban yang meyakinkan tapi salah
- **Knowledge cutoff** — tak tahu kejadian setelah waktu training
- ▶ Tak bisa membaca dokumen **internal** kantor/sekolahmu
- ▶ Untuk fakta terbaru / spesifik → jawaban tak bisa dipercaya

“berita minggu lalu?”



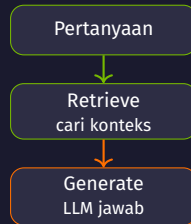
??? → menebak

LLM pintar **berbahasa**, tapi ingatannya **terbatas & beku** — dan kadang ngawur.

Solusi: RAG

Retrieval-Augmented Generation

- ▶ **RAG = Retrieval** (cari) + **Generation** (jawab)
- ▶ **Retrieve**: ambil potongan info relevan dari dokumenmu
- ▶ **Generate**: LLM menyusun jawaban dari **konteks** itu
- ▶ Hasilnya: faktual, bisa di-update, halusinasi **berkurang**
- ▶ Tambah pengetahuan = tambah dokumen — **tanpa latih ulang**



RAG tak mengubah otak LLM — ia **membekali** LLM konteks faktual segar sebelum menjawab.

Analogi: Ujian Buka Buku

Menjawab sambil melihat sumber, bukan dari ingatan saja

Tutup buku (LLM polos):

- ▶ Jawab dari ingatan → mudah lupa / salah

Buka buku (RAG):

- ▶ Boleh buka halaman relevan dulu
- ▶ **Retrieve** = menemukan halaman yang tepat
- ▶ Konteks dibekalkan ke LLM sebelum menjawab
- ▶ Jawaban bisa ditelusuri ke sumbernya

LLM
+ buku sumber
(knowledge base)

Dibekali konteks faktual dulu, baru menjawab — bukan tebakan dari ingatan.

Yang RAG Ubah — dan Tidak

Harapan yang realistis

RAG membantu:

- ✓ Jawaban berbasis dokumenmu
- ✓ Update info = update dokumen
- ✓ Bisa beri **sitasi** sumber
- ✓ Halusinasi **berkurang**

RAG bukan sihir:

- Konteks salah → jawaban tetap meleset
- Halusinasi tak **hilang** 100%
- Kualitas = sebaik retrieval-nya

“Garbage in, garbage out”: jawaban RAG hanya sebaik konteks yang berhasil ditemukan.

Act 2

Anatomi RAG

Embedding, pencarian makna, dan generator

Alur End-to-End

Tiga langkah inti: embed → retrieve → generate

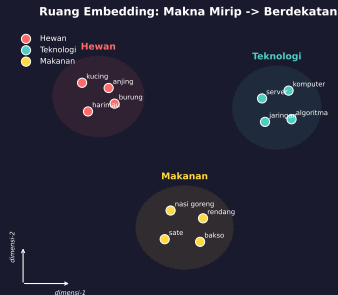


Pertanyaan di-**embed**, dicocokkan ke konteks, lalu LLM menjawab dari konteks itu.

Embeddings: Makna Jadi Angka

Teks diubah jadi vektor; makna mirip → letak berdekatan

- ▶ **Embedding**: teks → **vektor** (deretan angka)
- ▶ Kalimat **mirip makna** → vektor **berdekatan**
- ▶ Komputer membandingkan **jarak antar-angka**, bukan kata
- ▶ Model **multilingual** MiniLM → paham Bahasa Indonesia *dan* Inggris



Embedding multilingual = kunci pencarian **makna** lintas-bahasa (tanya ID, dokumen EN tetap ketemu).

Pencarian Makna, Bukan Kata

Beda dengan Ctrl+F / pencarian kata kunci

Keyword (lexical)

- ▶ Cocokkan **kata persis**
- ▶ “mobil” $\neq$ “kendaraan” (beda kata)
- ▶ Sinonim & parafrase lolos

Semantic (embedding)

- ▶ Cocokkan **makna**
- ▶ “mobil” $\approx$ “kendaraan” (vektor dekat)

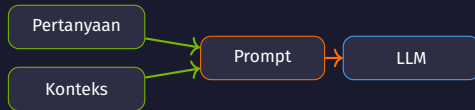


Semantic search menemukan dokumen relevan **walaupun pakai kata berbeda.**

Augment & Generate

Konteks disisipkan ke prompt, lalu LLM menjawab

- ▶ **Augment:** gabung **konteks** hasil retrieve + pertanyaan jadi satu prompt
- ▶ Format: bagian “Konteks: ...” lalu “Pertanyaan: ...”
- ▶ **Generate:** LLM menyusun jawaban **hanya dari konteks**
- ▶ Instruksi kunci: “bila tak ada di konteks, katakan tidak tahu” → tekan karangan



Augment = jembatan retrieve → generate: konteks + pertanyaan → satu prompt grounded.

Act 3

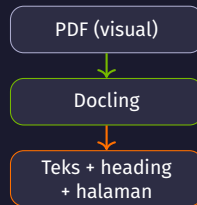
Dari Dokumen ke Potongan

Ingest dan chunking yang sadar-struktur

Ingest: PDF → Teks Terstruktur

Docling membaca dokumen seperti manusia

- ▶ PDF itu **visual** — copy-paste teks sering berantakan
- ▶ **Docling** memarsing **struktur**: heading, paragraf, **tabel**
- ▶ **OCR** otomatis untuk halaman hasil scan / gambar
- ▶ Simpan **nomor halaman & heading** tiap potongan
- ▶ Metadata itu → bahan baku **sitasi** nanti

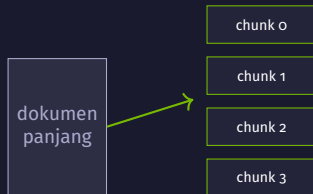


Ingest yang baik = retrieval yang baik: struktur & metadata terjaga sejak awal.

Kenapa Dipotong (Chunking)?

Dokumen panjang dipecah jadi potongan kecil

- ▶ Dokumen utuh **terlalu besar** untuk konteks LLM & embedding (batas token)
- ▶ Retrieval per-**potongan** jauh lebih **presisi** daripada per-dokumen
- ▶ Tiap chunk = satu unit makna yang bisa dicari & disitasi
- ▶ Ukuran chunk disesuaikan **batas token embedder** (mis. 512)



Chunk = unit retrieval. Terlalu besar → kabur; terlalu kecil → konteks hilang.

Strategi Chunking

Beberapa cara memotong, dengan trade-off masing-masing

Strategi	Cara	Sifat
Fixed (per-kata)	Potong tiap N kata + overlap	Sederhana, bisa putus kalimat
Sentence	Pak kalimat utuh sampai batas	Rapi, ukuran bervariasi
Semantic / struktur	Ikut paragraf & heading (Docling HybridChunker)	Sadar-konteks, terbaik untuk sitasi

HybridChunker memotong **ikut struktur** dokumen — dan menyimpan heading untuk sitasi.

Act 4

Pencarian yang Lebih Baik

Dua tahap: cepat dulu, akurat kemudian

Bi-encoder vs Cross-encoder

Dua cara mengukur relevansi — cepat vs akurat

Bi-encoder

- ▶ Embed query & dokumen **terpisah**
- ▶ Bandingkan vektor (cepat, bisa di-index)
- ▶ Cocok untuk **ribuan** dokumen
- ▶ Kurang presisi pada nuansa

Cross-encoder

- ▶ Baca query + dokumen **bersama**
- ▶ Jauh lebih **akurat** menilai relevansi
- ▶ Lambat — tak bisa di-index
- ▶ Hanya untuk **sedikit** kandidat

Bi-encoder cepat menyaring banyak; cross-encoder teliti menilai sedikit. Gabungkan keduanya!

Pencarian Dua Tahap

Over-fetch dengan bi-encoder, lalu rerank dengan cross-encoder

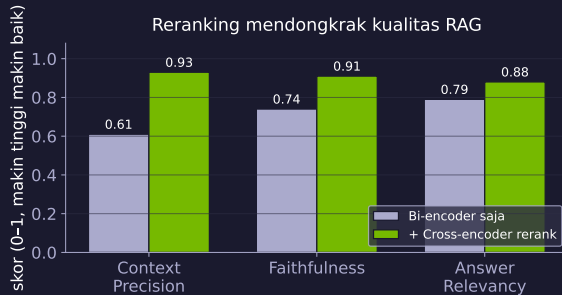


Ambil 20 kandidat cepat (bi-encoder) → urutkan ulang teliti (bge-reranker-v2-m3) → pakai 4 terbaik.

Dampak Reranking

Tahap kedua mendongkrak presisi konteks

- ▶ Reranker menilai **relevansi topik** dengan teliti
- ▶ Konteks lebih tepat → jawaban lebih **grounded**
- ▶ Biaya: sedikit lebih lambat per-query
- ▶ Jujur: reranker menilai **relevansi**, bukan jaminan jawaban ada di sana



angka ilustratif — arah selaras dengan pola nb03/nb04

Act 5

Apakah RAG-nya Bagus?

Mengukur kualitas dengan RAGAS

Empat Metrik RAGAS

Mengukur retrieval *dan* jawaban — bukan sekadar “terasa benar”

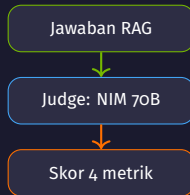
Metrik	Menilai	Pertanyaannya
Faithfulness	Jawaban	Apakah jawaban setia pada konteks (tak mengarang)?
Answer Relevancy	Jawaban	Apakah jawaban menjawab pertanyaan?
Context Precision	Retrieval	Apakah konteks yang diambil relevan ?
Context Recall	Retrieval	Apakah semua info yang dibutuhkan terambil?

Dua metrik menilai **retrieval**, dua menilai **jawaban** — ukur keduanya untuk tahu di mana yang lemah.

Siapa yang Menilai? LLM Judge

Model besar menilai jawaban model lain

- ▶ RAGAS pakai **LLM judge** untuk menilai tiap metrik
- ▶ Judge harus **cukup pintar** — model kecil sering gagal parsing JSON → skor NaN
- ▶ Kita pakai **NVIDIA NIM** (Llama-3.3-70B) sebagai judge → andal, tanpa GPU lokal
- ▶ Embedding metrik tetap lokal (multilingual MiniLM)



Judge yang lemah = pengukuran yang menyesatkan. Pakai model judge yang kuat.

Act 6

Menskalakan Index

Recall vs kecepatan saat data membesar

Taksonomi Index FAISS

Dari pencarian persis sampai perkiraan cepat

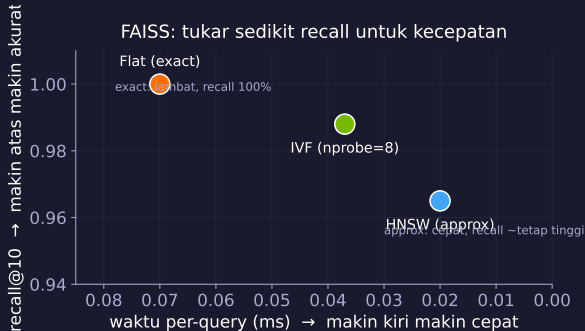
Index	Cara	Sifat
Flat	Bandingkan ke semua vektor	Recall 100%, lambat saat besar
IVF	Bagi ruang jadi sel, cari sebagian (n_{probe})	Cepat, recall sedikit turun
HNSW	Graf tetangga berlapis	Sangat cepat, perkiraan

Cosine vs L2: untuk cosine, **normalisasi** dulu lalu pakai inner-product (IndexFlatIP).

Trade-off: Recall vs Kecepatan

Index perkiraan menukar sedikit akurasi untuk kecepatan

- ▶ Data kecil → **Flat** sudah cukup (& 100% recall)
- ▶ Data besar → **IVF/HNSW** jauh lebih cepat
- ▶ Atur `nprobe` → geser recall vs kecepatan
- ▶ Simpan index ke disk (`write_index`) — muat saat start



Persistence & ChromaDB

Index yang awet, tak perlu dibangun ulang tiap start

- ▶ **FAISS:** `write_index` / `read_index` → simpan & muat dari disk
- ▶ **ChromaDB:** vector store dengan persistence otomatis (`PersistentClient`)
- ▶ Bawa vektor sendiri (`embedding_function=None`) → hindari unduh model ganda
- ▶ Pilih sesuai kebutuhan: FAISS (ringan, cepat) atau Chroma (metadata, filter)

Bangun index sekali, **simpan**, lalu muat saat start — server tak rebuild tiap permintaan.

Act 7

RAG yang Ingat

Percakapan multi-giliran

Masalah Pertanyaan Lanjutan

Elipsis & kata ganti membuat retrieval gagal

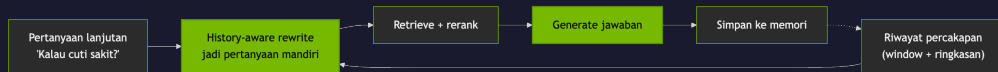
#	Pertanyaan	Masalah
1	“Berapa hari cuti tahunan?”	Aman — mandiri
2	“Kalau cuti sakit?”	Elipsis — “kalau” merujuk giliran 1
3	“Perlu surat dokter?”	Kata ganti implisit

Embed “Kalau cuti sakit?” apa adanya → terlalu ambigu → retrieval meleset.

Pertanyaan lanjutan harus dibuat **mandiri** dulu sebelum di-retrieve.

Memori + History-aware Rewriting

Tulis ulang jadi mandiri, lalu retrieve



Memori **window** (giliran terbaru) + **ringkasan** (giliran lama) → rewrite resolusi
“itu”/“-nya”.

Act 8

Jawaban yang Bisa Dipercaya

Sitasi dan verifikasi

Jawaban Bersitasi

Setiap klaim menunjuk sumbernya

- ▶ Tiap konteks diberi label [S1] .. [S4] + halaman/heading
- ▶ LLM diminta **membubuhkan** label [S#] pada klaim
- ▶ Pembaca bisa **menelusuri** klaim ke dokumen asli
- ▶ Ditampilkan: jawaban + daftar **“Lihat sumber”**

“Cuti sakit hingga 10 hari
[S1].”

Lihat sumber:

[S1] hlm 3 — Pasal Cuti

Sitasi mengubah jawaban dari “percaya saja” menjadi **“bisa diperiksa”**.

Verifikasi Sitasi

Jangan percaya label yang ditulis model begitu saja

- ▶ Model kecil bisa **mengarang** label (mis. [S5] padahal hanya ada 4)
- ▶ Kita **parse** label [S#] dari jawaban, cek dalam rentang yang sah
- ▶ Label valid ditandai ✓; yang di luar daftar ditandai △ “periksa!”
- ▶ Inilah inti capstone: jawaban yang **terverifikasi**, bukan sekadar terlihat rapi

“Trust, but verify”: tampilkan sumber **dan** cek labelnya secara otomatis.

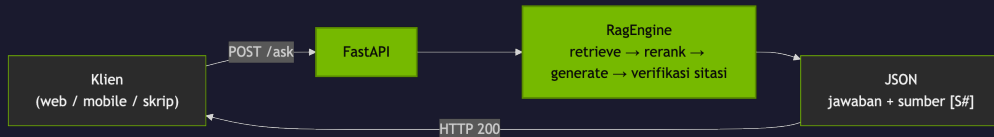
Act 9

Deploy

Dari notebook ke layanan /ask

RAG sebagai Layanan

Bungkus pipeline jadi API FastAPI /ask



Model dimuat **sekali** saat start; tiap permintaan /ask balas **JSON** jawaban + sumber (tervalidasi Pydantic).

Generator: Cloud vs Lokal

Satu sakelar, dua mode

Mode	Model	Kapan dipakai
NIM (default)	NVIDIA NIM, Llama-3.3-70B (cloud)	Andal, tanpa GPU — cocok untuk server
Qwen (lokal)	Qwen2.5-3B 4-bit (T4)	Penuh on-prem / ber-GPU, tanpa API

Model besar (cloud) memberi rewriting & jawaban lebih tajam — **kualitas naik seiring kapasitas model.**

Act 10

Ringkasan & Peta

Apa yang sudah dibangun, dan kapan memakai apa

Perjalanan Modul 05

Delapan konsep, satu pipeline RAG yang lengkap

#	Konsep	Inti
1	Fundamentals	embed → retrieve → generate
2	Ingest & chunk	Docling + chunking sadar-struktur
3	Rerank	bi-encoder → cross-encoder
4	Ukur (RAGAS)	4 metrik, judge NIM
5	Skala index	FAISS IVF/HNSW, ChromaDB
6	Conversational	memori + history-aware rewriting
7	Capstone	jawaban bersitasi terverifikasi
8	Deploy	layanan FastAPI /ask

Panduan Keputusan

Kapan memakai apa

Pertanyaan	Pegangan
Ukuran chunk?	Sesuai batas token embedder (mis. 512), ikut struktur
Perlu rerank?	Ya bila presisi konteks penting (hampir selalu)
Index apa?	Kecil: Flat. Besar: IVF/HNSW + persistensi
Generator apa?	Server tanpa GPU: NIM. On-prem GPU: lokal
Percaya jawaban?	Hanya bila ada sitasi & sitasi terverifikasi

RAG yang baik = retrieval tepat + jawaban grounded + sumber yang bisa ditelusuri.

Terima Kasih!

Pertanyaan? Diskusi?

RAG Fundamentals · Modul 05
Navasena Gen-ML Course